HLD

High-Level Design (HLD)

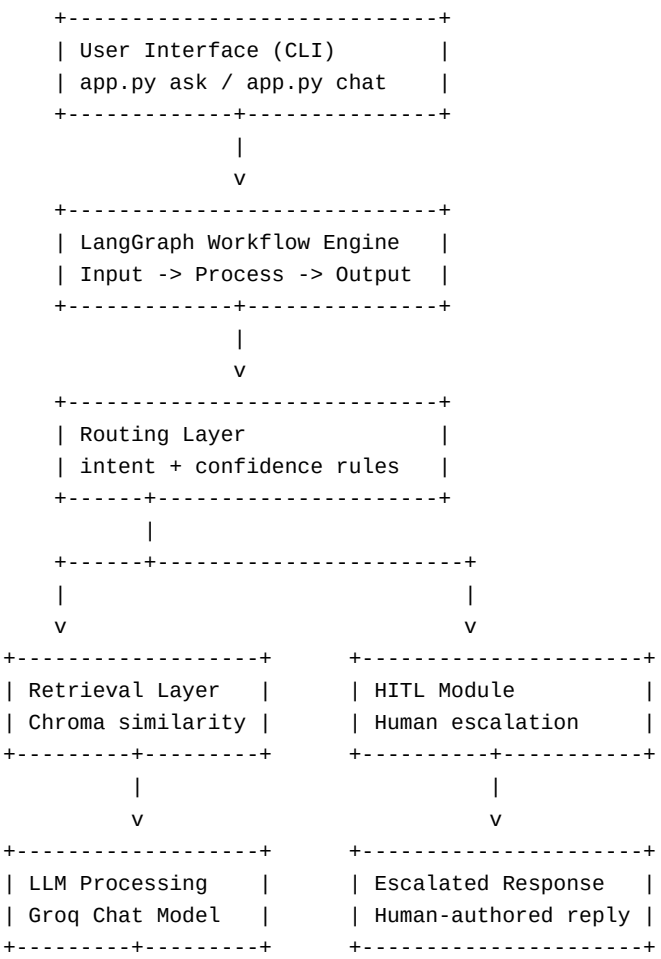
1. System Overview

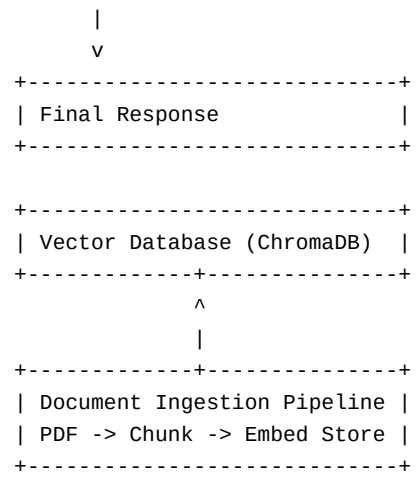
Problem Definition
Customer support operations often rely on large policy documents and runbooks. Traditional bots either use fixed rule trees (low coverage) or pure LLM responses (high hallucination risk). The required solution must ground answers in a PDF knowledge base and apply explicit workflow control with safe escalation.

Scope of the System
In scope:
- Ingest one or more support knowledge PDFs (current implementation: one main PDF)
- Convert PDF text into semantic chunks and store embeddings in ChromaDB
- Retrieve relevant chunks for each user query
- Run a graph-based decision workflow (LangGraph)
- Route to automatic answer or human escalation using deterministic criteria
- Support command-line interaction

Out of scope for current version:
- Multi-tenant auth and role-based access control
- Full ticketing platform integration
- Web UI and SSO
- Long-term conversation memory

2. Architecture Diagram (Mandatory)





3. Component Description

Document Loader
- Module: `src/rag_support/ingest.py`
- Uses `PyPDFLoader` to parse PDF pages into LangChain documents.

Chunking Strategy
- Recursive character splitter with overlap.
- Current defaults:
  - `chunk_size = 900`
  - `chunk_overlap = 150`
- Rationale: preserve policy sentence continuity while keeping chunks small enough for retrieval precision.

Embedding Model
- Local HuggingFace sentence-transformer model:
  - `sentence-transformers/all-MiniLM-L6-v2`
- No paid embedding API required.

Vector Store
- ChromaDB persistent local store (`chroma_db/`).
- Supports fast similarity search and easy local development.

Retriever
- Uses similarity search with vector distance converted to a normalized confidence score.
- Returns top-k chunks plus score metadata for routing confidence checks.

LLM
- Groq chat model (default: `llama-3.1-8b-instant`).
- Used only after retrieval and route decision.

Graph Workflow Engine
- LangGraph `StateGraph` with two operational nodes:
  - `process`
  - `output`
- Start and end are control boundary states.

Routing Layer
- Implemented in `src/rag_support/routing.py` and used by workflow.
- Detects intent and decides:

  - `auto_answer`
  - `escalate`

HITL Module
- Implemented in `src/rag_support/hitl.py`.
- CLI-based human response collection for escalated queries.

4. Data Flow

Ingestion Flow (PDF -> Vector Store)
1. Operator runs `python app.py ingest --pdf <path>`
2. PDF pages loaded
3. Pages chunked with overlap
4. Chunk embeddings generated
5. Chunks stored in Chroma collection

Query Lifecycle (Question -> Answer)
1. User asks a question via `ask` or `chat`
2. `process` node retrieves relevant chunks and confidence score
3. Intent detection runs over user query
4. Route decision runs on:
   - intent category
   - retrieval confidence
   - context presence
   - query complexity
5. `output` node behavior:
   - `auto_answer`: grounded answer from Groq using retrieved context
   - `escalate`: invoke human response workflow
6. Final response returned with debug context (intent, route, confidence)

5. Technology Choices

Why ChromaDB
- Local persistent vector database with low setup overhead
- Works well for medium-size support KB workloads
- Easy integration with LangChain retrievers

Why LangGraph
- Explicit, inspectable control flow for LLM systems
- Clear state transitions for production reasoning
- Supports deterministic routing + HITL patterns naturally

LLM Choice (Groq)
- Free-tier accessible API for development
- Fast inference for interactive support scenarios
- Good compatibility via `langchain-groq`

Additional Tools
- `Typer` for clean CLI commands
- `Rich` for readable terminal interactions
- `ReportLab` for PDF deliverable generation

6. Scalability Considerations

Large Documents
- Incremental ingestion by file sections (future extension)
- Metadata filters to restrict retrieval by policy domain
- Optional semantic chunking for better long-policy segmentation

Increasing Query Load
- Move from local CLI to service deployment with request queue
- Cache frequent query embeddings and retrieval results
- Horizontal scale by stateless API workers and shared vector store

Latency Concerns
- Primary latency contributors:
  - embedding generation at ingestion time
  - retrieval call
  - LLM generation
- Mitigations:
  - pre-ingest and warm vector index
  - tune `top_k` to balance quality/latency
  - use smaller/faster Groq model for low-priority channels

7. Operational Safety Notes

- The workflow escalates when confidence is low or query is sensitive.
- This protects against confident-but-wrong auto answers.
- Human override remains first-class in the architecture, not an afterthought.